Final Capstone Project Report

Algorithms at Scale – IMDb Movie Title Search Optimization

Name: Tulsi Mansukhbhai Hudka

NetID: xzm14

1. Project Overview

For this final capstone project, I used the IMDb Non-Commercial Dataset to study how different search algorithms perform as the dataset grows very large. The main focus of the project was to compare a simple baseline approach with a more optimized approach and analyze how both behave in terms of runtime and memory usage.

The problem I selected was movie title lookup. Given a movie title, the program searches the dataset and returns matching records. Even though this sounds simple, the IMDb dataset contains a very large number of records, which makes it a good choice for studying algorithmic scaling and performance differences.

The baseline approach uses linear search, which scans every record one by one until it finds the target title. The optimized approach uses a hash-table-based index that allows near constant-time lookups.

The goal of this project was not only to implement both approaches, but also to benchmark them across multiple dataset sizes and connect the experimental results back to Big-O analysis.

2. Dataset

The dataset used in this project is the IMDb Non-Commercial Dataset.

File used:

- title.basics.tsv

This dataset contains information about movies and TV titles, including:

- Movie title
- Year
- Genre
- Unique identifier

The dataset is large enough to clearly demonstrate the difference between O(n) and O(1) style operations.

The following input sizes were tested:

- 10,000 records
- 50,000 records
- 100,000 records

- 500,000 records
- 1,000,000 records

3. Baseline Approach

The baseline implementation uses linear search.

In this method, the program scans every row in the dataset and compares the movie title with the target query.

Steps

1. Load the dataset into memory using pandas.
2. Iterate through every record.
3. Compare each title with the target title.
4. Return matching results.

Time Complexity

The time complexity of linear search is:

O(n)

because the algorithm may need to examine every record in the dataset.

Advantages

- Simple to implement
- Uses very little additional memory

Disadvantages

- Slow for very large datasets
- Runtime increases linearly as dataset size grows

4. Optimized Approach

The optimized implementation uses a hash table (Python dictionary).

Instead of scanning the dataset every time a query is made, the program first builds an index where:

movie title → list of matching records

Once the index is created, movie lookups become extremely fast.

Steps

1. Read dataset records.
2. Build hash table index.
3. Store movie titles as dictionary keys.
4. Perform direct lookup using dictionary access.

Time Complexity

Index Building

O(n)

because each record must be processed once.

Query Time

Approximately:

O(1)

because dictionary lookup is nearly constant time.

Advantages

- Extremely fast search performance
- Runtime remains almost constant even as dataset size increases

Disadvantages

- Requires additional memory to store the index
- Has an upfront preprocessing cost

5. Benchmarking Methodology

To compare both approaches fairly, benchmarking was performed using five different dataset sizes.

The benchmark script measures:

- Baseline query runtime
- Optimized query runtime
- Hash index build time
- Memory usage
- Correctness verification

The Python time module was used for runtime measurement.

The Python tracemalloc module was used for memory analysis.

The program also verifies that both algorithms produce identical outputs.

For every benchmark:

Outputs match: True

was confirmed.

6. Benchmark Results

Runtime Results

The experimental results showed a very large performance improvement when using the optimized hash-table approach.

Example Results

Dataset Size	Baseline Runtime	Optimized Runtime	Speedup
10K	0.018 sec	0.000054 sec	341x
50K	0.094 sec	0.000075 sec	1268x
100K	0.191 sec	0.000125 sec	1528x
500K	0.984 sec	0.000273 sec	3609x
1M	2.024 sec	0.000671 sec	3016x

The results clearly show that the baseline runtime increases rapidly as the dataset grows, while the optimized query time stays extremely small.

7. Memory Usage Analysis

The optimized approach achieved major runtime improvements, but it also required more memory.

This happened because the hash-table index stores additional data structures in memory.

At 1 million records:

- Baseline approach used very little additional memory
- Hash indexing used over 100 MB of memory

This demonstrates an important algorithmic tradeoff:

Faster query performance often requires additional memory usage.

8. Theory vs Practice

The experimental results closely matched the theoretical Big-O expectations.

Baseline Linear Search

The runtime increased roughly linearly with dataset size.

This matches the theoretical complexity:

O(n)

because the algorithm scans records one by one.

Hash Table Search

The optimized query time remained nearly constant.

This matches the expected:

O(1)

lookup behavior of hash tables.

The benchmark results clearly demonstrate how asymptotic complexity becomes more important as dataset size grows.

9. Amortized Cost Analysis

One important observation from this project is that the optimized approach includes an upfront preprocessing cost.

Before queries can be performed, the hash-table index must first be built.

Building the index takes:

O(n)

However, once the index exists, repeated searches become extremely fast.

This means the preprocessing cost becomes amortized over multiple queries.

For applications that perform many searches, the optimized approach becomes significantly more efficient overall.

This was one of the main lessons learned from the project.

10. Challenges Encountered

The biggest challenge during this project was working with the large IMDb dataset.

Initially, GitHub rejected the repository because the dataset file was larger than GitHub's upload limit.

To solve this problem:

- The dataset was excluded using .gitignore
- Dataset setup instructions were added to the README
- Only the code and generated results were included in the repository

Another challenge was ensuring that benchmarking actually used the full dataset instead of accidentally running on a small sample file.

Once the correct dataset path was fixed, the benchmark results became much more realistic and meaningful.

11. Future Improvements

If I had more time to continue this project, I would add:

- Partial title matching

- Trie-based autocomplete search
- Multi-field searching (title + genre + year)
- More advanced indexing structures
- Parallel processing for index construction

I would also like to compare additional algorithms such as trie structures and inverted indices.

12. Reflection

Before running the benchmarks, I expected the optimized approach to be faster, but I did not expect the speedup to become several thousand times faster at larger scales.

One interesting observation was that the memory cost of the hash-table index also became very large.

This project helped me better understand the practical tradeoff between runtime efficiency and memory usage.

It also showed how theoretical algorithm complexity directly affects real-world system performance when working with large datasets.

13. Conclusion

This project successfully demonstrated the difference between a baseline linear search approach and an optimized hash-table-based search approach on a large real-world dataset.

The baseline algorithm was simple but scaled poorly as dataset size increased.

The optimized approach required additional preprocessing and memory usage, but achieved dramatically faster query performance.

The benchmark results strongly supported the theoretical Big-O analysis and clearly showed the importance of choosing efficient algorithms for large-scale systems.

Overall, this project provided practical experience with benchmarking, scalability analysis, memory tradeoffs, and real-world algorithm optimization.